

Pesquisa: Fundamentos de Dart e POO

1. Funções em Dart

Funções são blocos de código criados para realizar tarefas específicas. Elas ajudam a organizar o programa, evitar repetição e reutilizar funcionalidades.

Uma função pode receber **parâmetros**, realizar um processamento e retornar um resultado. Dart também possui parâmetros opcionais, parâmetros nomeados e funções anônimas.

As funções são utilizadas em cálculos, validações, processamento de dados e execução de ações.

2. Listas

Listas são estruturas utilizadas para armazenar vários valores em uma sequência ordenada. Cada elemento possui uma posição chamada índice, começando normalmente pelo zero.

São utilizadas para armazenar informações como nomes, produtos, notas e tarefas. Dart permite adicionar, remover, pesquisar, percorrer e transformar elementos de uma lista.

3. Maps

Maps armazenam informações no formato **chave e valor**. Cada chave identifica um determinado valor.

São muito utilizados para representar dados estruturados, configurações e informações recebidas de APIs, especialmente dados no formato JSON.

4. Programação Orientada a Objetos

A Programação Orientada a Objetos, ou POO, é um modelo de programação baseado em **classes e objetos**. Dart é uma linguagem orientada a objetos.

Os principais conceitos são classes, objetos, atributos, métodos, construtores, encapsulamento, herança, polimorfismo e abstração.

5. Classes e Objetos

Uma **classe** é um modelo que define características e comportamentos de uma entidade.

Um **objeto** é uma instância criada a partir de uma classe.

Os **atributos** representam as características ou dados de um objeto, enquanto os **métodos** representam as ações que ele pode realizar.

Por exemplo, uma classe que representa uma pessoa pode possuir informações como nome e idade e comportamentos relacionados a essa pessoa.

6. Construtores

Construtores são utilizados para inicializar objetos quando eles são criados.

Dart permite diferentes formas de construtores:

- **Construtor padrão:** utilizado como forma principal de criação do objeto.
- **Construtor nomeado:** permite criar objetos de maneiras alternativas dentro da mesma classe.
- **Parâmetros nomeados:** identificam os valores pelo nome, tornando a criação do objeto mais clara e organizada.

7. Encapsulamento

Encapsulamento é o conceito de proteger e controlar o acesso aos dados internos de uma classe.

Ele evita que informações importantes sejam modificadas de maneira inadequada e permite que a classe controle como seus dados são utilizados.

O encapsulamento ajuda a melhorar a segurança, organização e manutenção do programa.

8. Getters e Setters

Getters permitem acessar ou obter informações de maneira controlada.

Setters permitem modificar informações de maneira controlada, podendo realizar validações antes de aceitar um novo valor.

Por exemplo, uma classe pode impedir que uma idade negativa seja armazenada ou que o saldo de uma conta seja alterado diretamente.